

ARIMA

March 26, 2026

0.1 ARIMA model to forecast Colon Cancer

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from sklearn.metrics import mean_absolute_error, mean_squared_error
import itertools
```

```
[8]: CSV_PATH = '../data/seer_colorectal_incidence.csv'
df = pd.read_csv(CSV_PATH)
print(f'Shape: {df.shape}')
df.columns.tolist()
df.head()
```

Shape: (31, 3)

```
[8]:
```

	Year	Incidence_Rate	Death_Rate
0	1992	56.01	23.69
1	1993	54.82	23.39
2	1994	53.80	22.99
3	1995	52.45	22.64
4	1996	53.22	21.91

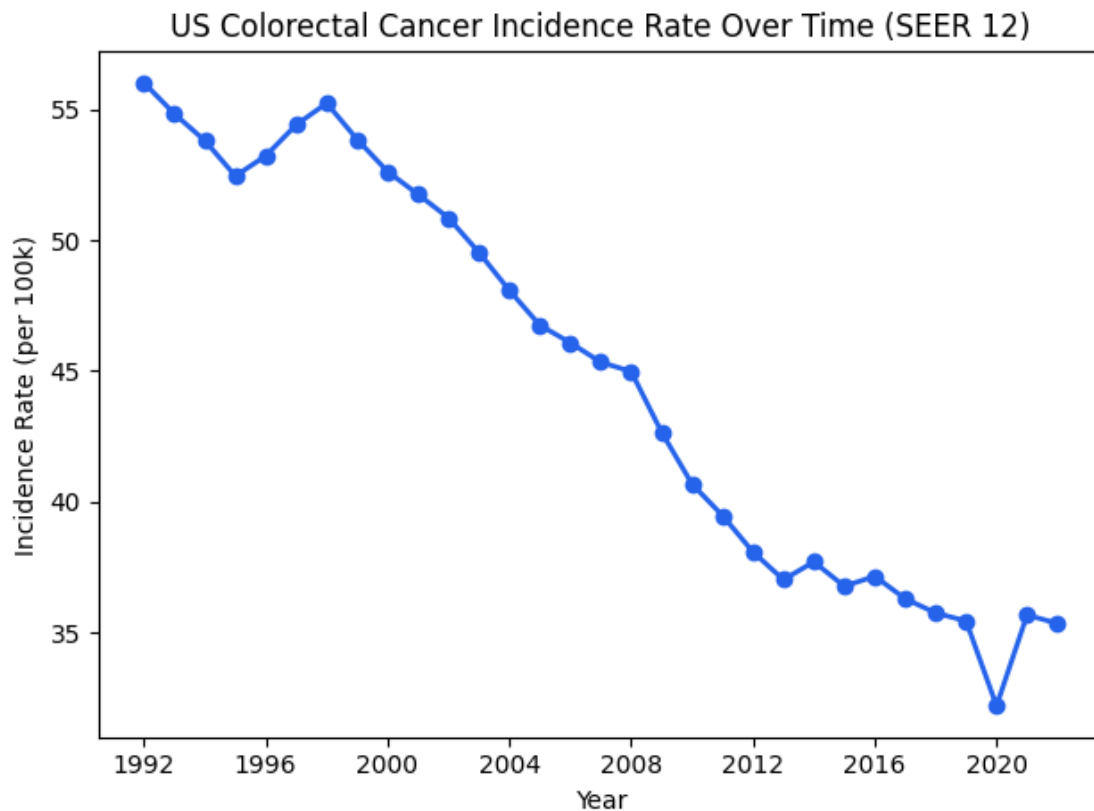
```
[9]: COL_YEAR = 'Year'
COL_RATE = 'Incidence_Rate'
COL_EXOG = 'Death_Rate'

ts = df.set_index('Year')
ts.index = pd.to_datetime(ts.index, format='%Y')
ts.index.freq = 'YS'
ts
```

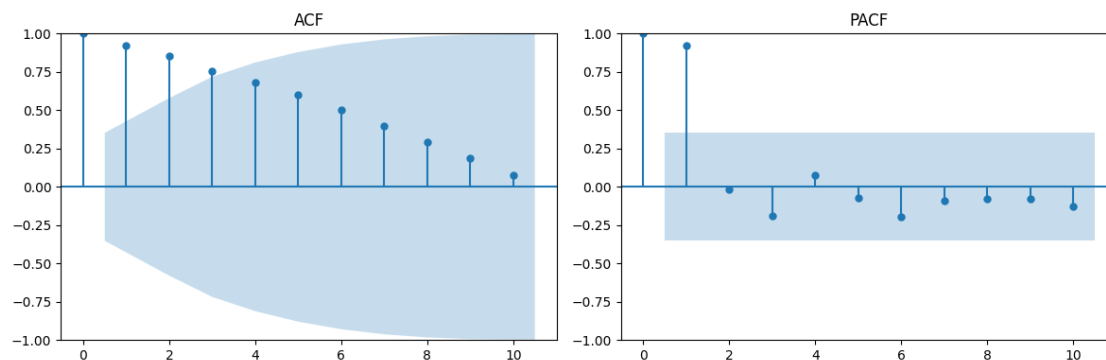
```
[9]:
```

	Incidence_Rate	Death_Rate
Year		
1992-01-01	56.01	23.69
1993-01-01	54.82	23.39
1994-01-01	53.80	22.99
1995-01-01	52.45	22.64
1996-01-01	53.22	21.91
1997-01-01	54.41	21.51
1998-01-01	55.23	21.22
1999-01-01	53.85	20.97
2000-01-01	52.62	20.71
2001-01-01	51.75	20.19
2002-01-01	50.85	19.80
2003-01-01	49.53	19.19
2004-01-01	48.08	18.13
2005-01-01	46.75	17.60
2006-01-01	46.08	17.32
2007-01-01	45.34	16.94
2008-01-01	44.98	16.49
2009-01-01	42.66	15.83
2010-01-01	40.68	15.55
2011-01-01	39.48	15.18
2012-01-01	38.09	14.80
2013-01-01	37.03	14.61
2014-01-01	37.71	14.28
2015-01-01	36.78	14.20
2016-01-01	37.16	13.90
2017-01-01	36.30	13.72
2018-01-01	35.76	13.40
2019-01-01	35.45	13.09
2020-01-01	32.21	12.87
2021-01-01	35.68	12.93
2022-01-01	35.36	12.76

```
[10]: fig, ax = plt.subplots()
ax.plot(ts.index, ts['Incidence_Rate'], marker='o', linewidth=2,
        color='#2563EB')
ax.set_title('US Colorectal Cancer Incidence Rate Over Time (SEER 12)')
ax.set_xlabel('Year')
ax.set_ylabel('Incidence Rate (per 100k)')
plt.tight_layout()
plt.show()
```



```
[11]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
      plot_acf(ts['Incidence_Rate'], ax=axes[0], lags=10, title='ACF')
      plot_pacf(ts['Incidence_Rate'], ax=axes[1], lags=10, title='PACF')
      plt.tight_layout()
      plt.show()
```



ACF before differencing shows a clear downward trend.

```
[12]: adf_result = adfuller(ts['Incidence_Rate'], autolag='AIC')
print(f'ADF Statistic : {adf_result[0]:.4f}')
print(f'p-value       : {adf_result[1]:.4f}')
print(f'Stationary     : {adf_result[1] < 0.05}')

```

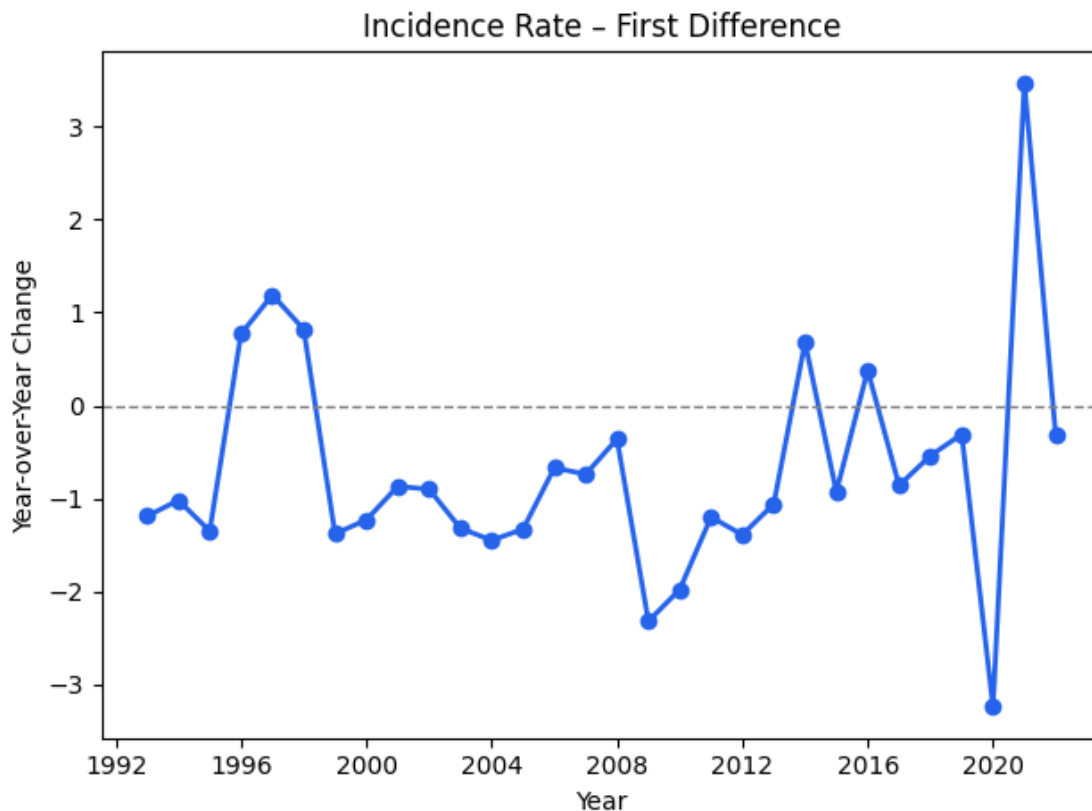
```
ADF Statistic : -0.6608
p-value       : 0.8566
Stationary    : False

```

```
[13]: ts_diff = ts['Incidence_Rate'].diff().dropna()

fig, ax = plt.subplots()
ax.plot(ts_diff.index, ts_diff, marker='o', linewidth=2, color='#2563EB')
ax.set_title('Incidence Rate - First Difference')
ax.set_xlabel('Year')
ax.set_ylabel('Year-over-Year Change')
ax.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.tight_layout()
plt.show()

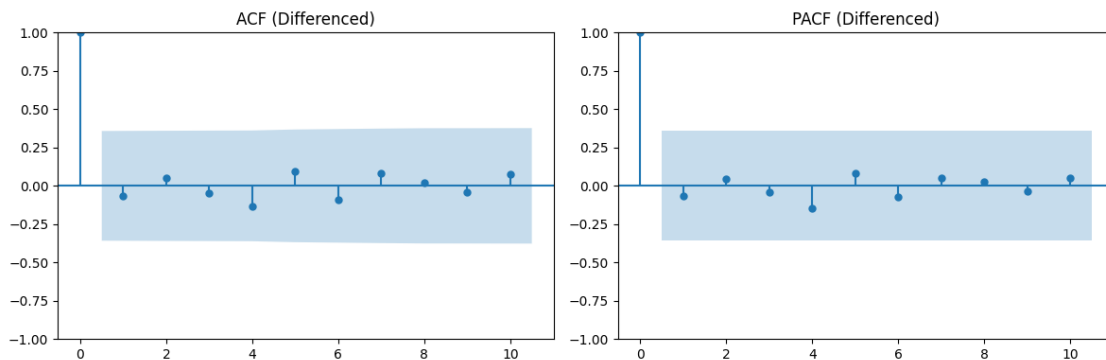
```



```
[14]: adf_diff = adfuller(ts_diff, autolag='AIC')
print(f'ADF Statistic : {adf_diff[0]:.4f}')
print(f'p-value       : {adf_diff[1]:.4f}')
print(f'Stationary     : {adf_diff[1] < 0.05}')
```

```
ADF Statistic : -5.5638
p-value       : 0.0000
Stationary     : True
```

```
[15]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(ts_diff, ax=axes[0], lags=10, title='ACF (Differenced)')
plot_pacf(ts_diff, ax=axes[1], lags=10, title='PACF (Differenced)')
plt.tight_layout()
plt.show()
```



```
[18]: TEST_YEARS = 5

train = ts.iloc[:-TEST_YEARS]
test  = ts.iloc[-TEST_YEARS:]

print(f'Train: {train.index[0].year} - {train.index[-1].year} ({len(train)}_
↪obs)')
print(f'Test:  {test.index[0].year} - {test.index[-1].year}  ({len(test)}_
↪obs)')
```

```
Train: 1992 - 2017 (26 obs)
Test:  2018 - 2022 (5 obs)
```

```
[20]: aic_results = []

for p, q in itertools.product(range(5), range(5)):
    try:
        m = ARIMA(train['Incidence_Rate'], order=(p, 1, q)).fit()
        aic_results.append({'p': p, 'd': 1, 'q': q, 'AIC': m.aic})
```

```
except Exception:
    continue
```

```
aic_df = pd.DataFrame(aic_results).sort_values('AIC').reset_index(drop=True)
print('Top 5 ARIMA orders:')
aic_df.head()
```

```
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
```

Top 5 ARIMA orders:

```
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
/home/ql2004/.local/lib/python3.10/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check
mle_retvals
```

```
warnings.warn("Maximum Likelihood optimization failed to "
```

```
[20]:      p  d  q      AIC
      0  1  1  0  70.577086
      1  2  1  1  71.475094
      2  2  1  0  72.136471
      3  1  1  1  72.162966
      4  0  1  2  72.652520
```

```
[21]: best_order = (0, 1, 1)
      print(f'Best order: ARIMA{best_order}')
```

Best order: ARIMA(0, 1, 1)

```
[22]: arima_model = ARIMA(train['Incidence_Rate'], order=best_order).fit()
      print(arima_model.summary())
```

```

                                SARIMAX Results
=====
Dep. Variable:      Incidence_Rate      No. Observations:      26
Model:              ARIMA(0, 1, 1)      Log Likelihood          -35.838
Date:              Thu, 26 Mar 2026      AIC                      75.677
Time:              17:12:38              BIC                      78.114
Sample:            01-01-1992            HQIC                     76.353
                  - 01-01-2017
Covariance Type:      opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ma.L1          0.4401      0.228        1.933      0.053      -0.006      0.886
sigma2          1.0208      0.426        2.394      0.017       0.185      1.856
=====
===
Ljung-Box (L1) (Q):      0.32      Jarque-Bera (JB):
1.32
Prob(Q):      0.57      Prob(JB):
0.52
Heteroskedasticity (H):      1.06      Skew:
0.56
Prob(H) (two-sided):      0.94      Kurtosis:
2.90
=====
===
```

Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

```
[23]: arima_forecast_obj = arima_model.get_forecast(steps=len(test))
arima_pred = arima_forecast_obj.predicted_mean
arima_ci = arima_forecast_obj.conf_int(alpha=0.05)

arima_pred.index = test.index
arima_ci.index = test.index
```

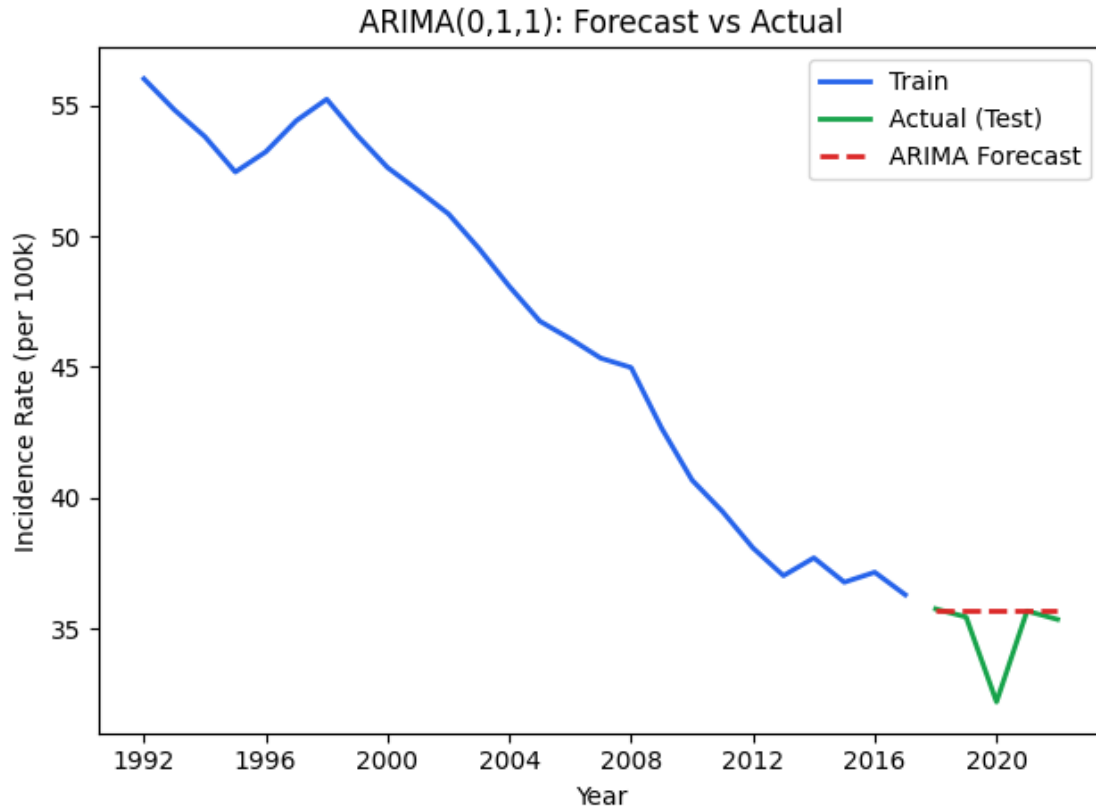
```
[24]: arima_mae = mean_absolute_error(test['Incidence_Rate'], arima_pred)
arima_rmse = np.sqrt(mean_squared_error(test['Incidence_Rate'], arima_pred))
arima_mape = np.mean(np.abs((test['Incidence_Rate'].values - arima_pred.values)
    ↪ / test['Incidence_Rate'].values)) * 100

print(f'MAE = {arima_mae:.4f}')
print(f'RMSE = {arima_rmse:.4f}')
print(f'MAPE = {arima_mape:.2f}%')
```

```
MAE = 0.8519
RMSE = 1.5897
MAPE = 2.60%
```

```
[26]: fig, ax = plt.subplots()
ax.plot(train.index, train['Incidence_Rate'], color='#2563EB', linewidth=2,
    ↪ label='Train')
ax.plot(test.index, test['Incidence_Rate'], color='#16A34A', linewidth=2,
    ↪ label='Actual (Test)')
ax.plot(arima_pred.index, arima_pred, color='#DC2626', linewidth=2,
    ↪ linestyle='--', label='ARIMA Forecast')

ax.set_title('ARIMA(0,1,1): Forecast vs Actual')
ax.set_xlabel('Year')
ax.set_ylabel('Incidence Rate (per 100k)')
ax.legend()
plt.tight_layout()
plt.show()
```

```
[27]: # Re-fit on ALL data
final_model = ARIMA(ts['Incidence_Rate'], order=best_order).fit()

# Forecast 10 years ahead
future_obj = final_model.get_forecast(steps=10)
future_pred = future_obj.predicted_mean
future_ci    = future_obj.conf_int(alpha=0.05)

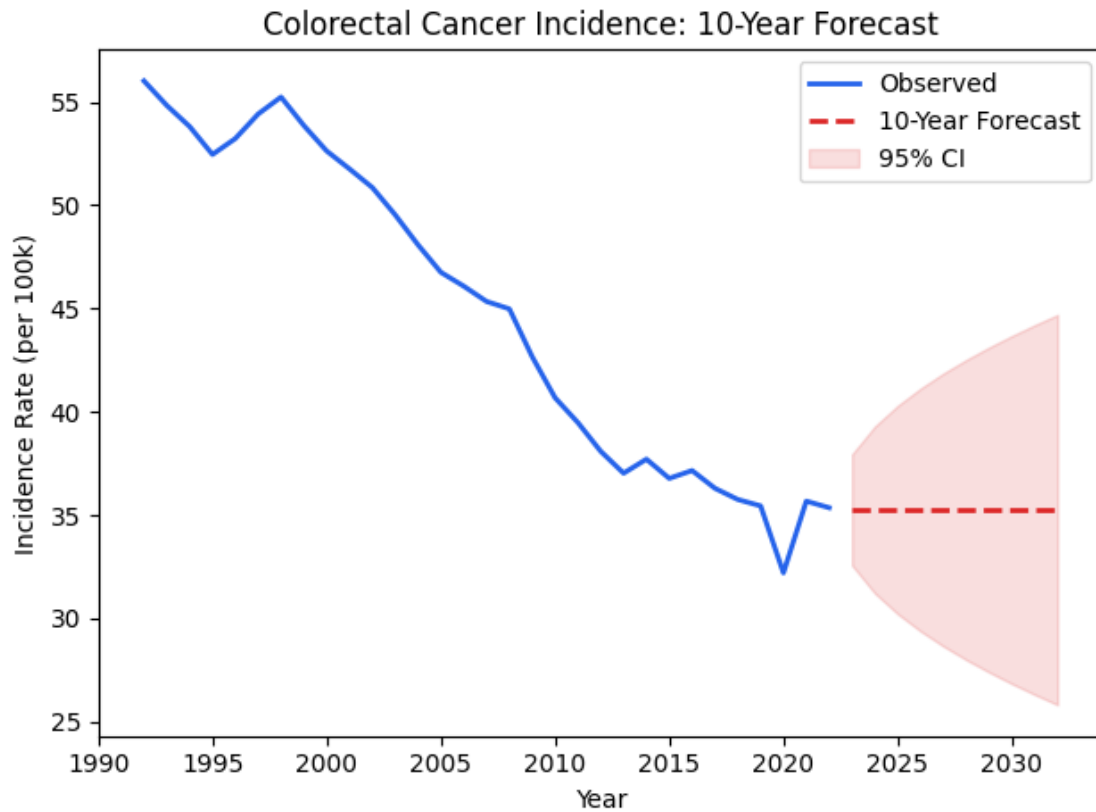
last_year   = ts.index[-1].year
future_idx = pd.date_range(start=str(last_year + 1), periods=10, freq='YS')
future_pred.index = future_idx
future_ci.index   = future_idx

[28]: fig, ax = plt.subplots()
ax.plot(ts.index, ts['Incidence_Rate'], color='#2563EB', linewidth=2,
        label='Observed')
ax.plot(future_pred.index, future_pred, color='#DC2626', linewidth=2,
        linestyle='--', label='10-Year Forecast')
ax.fill_between(future_pred.index, future_ci.iloc[:, 0], future_ci.iloc[:, 1],
                color='#DC2626', alpha=0.15, label='95% CI')
ax.set_title('Colorectal Cancer Incidence: 10-Year Forecast')
```

```

ax.set_xlabel('Year')
ax.set_ylabel('Incidence Rate (per 100k)')
ax.legend()
plt.tight_layout()
plt.show()

```



```

[29]: # Summary table
future_summary = pd.DataFrame({
    'Year': future_idx.year,
    'Predicted_Rate': future_pred.values.round(2),
    'CI_Lower': future_ci.iloc[:, 0].values.round(2),
    'CI_Upper': future_ci.iloc[:, 1].values.round(2)
})
future_summary

```

```

[29]:   Year  Predicted_Rate  CI_Lower  CI_Upper
0  2023             35.26      32.58      37.95
1  2024             35.26      31.23      39.30
2  2025             35.26      30.23      40.30
3  2026             35.26      29.39      41.14
4  2027             35.26      28.66      41.86

```

5	2028	35.26	28.01	42.52
6	2029	35.26	27.41	43.12
7	2030	35.26	26.85	43.68
8	2031	35.26	26.33	44.20
9	2032	35.26	25.83	44.70